

DEFENSA DEL PROYECTO

En este proyecto desarrollé un programa estadístico en Python capaz de calcular ciertas medidas tales como la media y la varianza muestrales, así como los intervalos de confianza de la media y varianza poblacionales. Además, como adición el programa incluye un menú interactivo con el cual elegir que cálculos hacer y automáticamente exporta la tabla ingresada y sus datos poblacionales a un Excel.

Implementación de Módulos, Métodos y Funciones:

Desde el inicio decidí estructurar el proyecto de forma modular, teniendo el programa cuatro, `main.py` con la implementación de todas las funciones creadas en los otros módulos, `tabla.py` donde están todas las funciones relacionadas a la creación de la tabla y el embellecimiento de esta con `tabulate`, `stats.py` donde aparecen todas las funciones para calcular las medidas estadísticas, y por último `export.py` en el cual aparecen las funciones relacionadas a el export de la tabla y de sus medidas a Excel.

En todos los módulos se separa la lógica en funciones específicas, con una única responsabilidad cada una, cada una de estas está diseñada para ser reusable, aislando lógica matemática evitando código duplicado, y facilitando futuras modificaciones.

Uso adecuado de docstrings y comentarios

Cada función de los diferentes módulos incluye docstrings siguiendo el estándar Google/PEP257, explicando que hace o que retorna

Buenas prácticas para código limpio, modular y legible

Las decisiones de diseño que apliqué incluyen:

- Funciones pequeñas y con una única responsabilidad.
- Nombres descriptivos de variables y funciones.
- Evitar lógica compleja dentro del main más allá de el flujo del programa con el menú interactivo .
- Evitar duplicación de código.
- Manejo de errores con `try except` y `while true` en los diferentes inputs para salidas limpias.
- Menú principal que centraliza la interacción del usuario.

Ejemplo de buenas prácticas:

- Funciones puras para cálculos estadísticos.
- Separación de cálculos vs interfaz.
- Flujo principal (`ejecutar_flujo_principal`) que organiza el proceso.

La modularización permite fácil mantenimiento, escalabilidad y análisis independiente de cada componente.

Aplicación del Zen de Python y PEP-8

Durante el desarrollo identifiqué principios del **Zen of Python** que se aplicaron de forma directa:

- “Simple es mejor que complejo”:
Funciones pequeñas, con pocas líneas y bajo acoplamiento.
- “La legibilidad cuenta”:
Código auto explicativo, estilo PEP8, nombres significativos.
- “Debería haber una—y preferentemente una—obvia manera de hacerlo”:
Se priorizó un único camino de flujo, guiado por el menú.
- “escaso es mejor que denso”:
Evité código poco legible o con operaciones inline innecesarias.

Además, el estilo cumple con **PEP8**:

- snake_case en funciones.
- Líneas cortas (<80 chars).
- Espacios alrededor de operadores.
- Imports ordenados.
- Código estructurado en bloques lógicos.

Uso del entorno virtual

El uso de un entorno virtual en este programa es muy importante ya que trabaja con cuatro librerías diferentes, scypy, tabulate, numpy, y openpyxl, y al ser tantas esto podría ocasionar incompatibilidades con otras librerías que el usuario podría tener instalado en su ordenador.

El hecho de usar un entorno virtual también hace que el proyecto sea portátil, reproducible y escalable, cualidades necesarias para desarrollo colaborativo.

Contenido del archivo README.md

Si bien el programa es bastante intuitivo de usar, el archivo README.md fue incluido para permitir que cualquier usuario o desarrollador entienda rápidamente cual es el flujo del programa, entienda para que sirve y sepa como utilizarlo correctamente.

A su vez es importante ver el archivo requirements.txt para saber que librerías se deben instalar en el entorno virtual.